💧 Rebase vs Merge (nível profissional de Git)

Se você entender isso de verdade, você já pensa como engenheiro.

🧠 Problema que isso resolve

Quando você trabalha com branches, chega uma hora que precisa:

👉 juntar mudanças

👉 manter o histórico organizado

E existem **duas formas principais**:

- `merge`
- `rebase`

🔀 1. `git merge` — juntar mantendo o histórico

📌 Comando:

```
git merge outra-branch
```

🧠 O que ele faz?

👉 Junta duas linhas **sem apagar o histórico original**

🌳 Visual

Antes:

```
main:   A — B
             \
login:        C — D
```

Depois do merge:

```
main:   A — B ——— M
             \    /
              C — D
```

👉 M = commit de merge

💡 Característica

- histórico completo
- mais “bagunçado”
- mais seguro

☑ Quando usar?

- trabalho em equipe
- histórico importante
- projetos grandes

🧬 2. `git rebase` — reescrever o histórico

📌 Comando:

```
git rebase main
```

🧠 O que ele faz?

👉 Ele “move” sua branch para cima da outra

🌳 Visual

Antes:

```
main:   A — B
             \
login:        C — D
```

Depois do rebase:

```
main:   A — B — C' — D'
```

👉 C e D são recriados (C', D')

💡 Característica

- histórico limpo
- linear
- mais difícil de entender

⚔️ Diferença direta (ESSENCIAL)

Merge 🧩	Rebase 🔬
mantém histórico	reescreve histórico
cria commit extra	não cria merge commit
mais seguro	mais “perigoso”
mais bagunçado	mais limpo

🧠 Analogia simples

- **merge** → junta duas histórias 📚
- **rebase** → reescreve a história ✍️

⚠️ REGRA DE OURO (MUUUUITO IMPORTANTE)

❌ Nunca use rebase em código que já foi enviado (push)

👉 Isso pode quebrar o projeto de outras pessoas

💧 Exemplo real (seu cenário)

Você criou uma feature:

```
git switch -c pagamento
```

Enquanto isso, a `main` mudou.

🔀 Com merge:

```
git switch pagamento
git merge main
```

👉 resolve e continua

🍬 Com rebase:

```
git switch pagamento
git rebase main
```

👉 sua branch fica “como se tivesse começado depois”

💥 Conflitos no rebase

👉 também acontecem

Quando acontece:

```
CONFLICT
```

Resolver:

1. editar arquivos

2. depois:

```
git add .
git rebase --continue
```

Cancelar:

```
git rebase --abort
```

🧠 Quando usar cada um (visão profissional)

☑ Use MERGE quando:

- está em equipe
- quer segurança
- histórico importa

☑ Use REBASE quando:

- quer histórico limpo
- está trabalhando sozinho
- antes de fazer push

💧 Fluxo profissional real

```
git checkout feature

git fetch
git rebase main
```

```
git push
```

🧠 Nível engenharia (visão profunda)

Merge:

👉 preserva a história real

Rebase:

👉 cria uma história ideal

⚠️ Erros comuns

❌ usar rebase sem entender

→ quebra histórico

❌ misturar tudo sem padrão

→ projeto vira caos

🚀 O que você aprendeu aqui

Agora você sabe:

- como integrar código corretamente
- como manter histórico limpo
- como evitar problemas em equipe

💧 Próximo nível (última peça importante)

👉 **Stash, Reset e Restore (controle total do código)**

Isso é tipo:

“voltar no tempo, apagar, salvar temporariamente”